

Finger-Vein Biometric Recognition: A Comparative Study of ResNet-18 and VGG-16

Giuseppe Borracchi, 1983263, *Sapienza University of Rome*,
Robert Cristian Iacobus, 1834884, *Sapienza University of Rome*,

Abstract—This project presents the design and evaluation of a finger vein-based biometric recognition system using deep learning models such as ResNet-18 and VGG-16. Unlike traditional systems, images are preprocessed by identifying finger boundaries to extract normalized Regions of Interest (ROIs). These images serve as input to convolutional models for the extraction of 512-dimensional feature vectors (embeddings), learned through the ArcFace loss function. The system is trained and evaluated on the MMCBNU_6000 dataset using a Two-Fold Cross-Validation architecture, evaluating the calculation of cosine distances and metrics such as EER (Equal Error Rate), ROC, AUC and DET. The results reflect the applicability of deep learning architectures for extracting stable information in finger vein biometric recognition.

Index Terms—Biometrics, Deep Learning, Finger-vein Recognition, Feature Extraction, ArcFace, Two-Fold Cross-Validation Architecture, Evaluation

1 INTRODUCTION

In this work, we analyze and develop the performance of models such as ResNet-18 and VGG-16 to generate embeddings in the field of biometric finger vein recognition. Finger vein-based recognition has attracted considerable attention because of its numerous advantages: the vein patterns are unique, permanent, difficult to replicate, and allow contactless detection of the subject's vitality.

The experiments for this project were conducted using the MMCBNU_6000 dataset. This public database contains 6,000 samples provided by 100 volunteers of different nationalities, with 10 images acquired for each finger.

2 DATASET

The experiments presented in this work were conducted on the MMCBNU_6000 finger vein database [5], a publicly available benchmark collected by the Division of Electronics and Information Engineering at Chonbuk National University (South Korea) and first introduced by Lu et al. at the 2013 International Congress on Image and Signal Processing (CISP). The database was designed to provide a large-scale, demographically diverse benchmark for finger vein recognition research, and it has since become one of the most widely used datasets in the field, together with other public collections such as FV-USM, SDUMLA-HMT, and HKPU-FV.

The database was collected from 100 subjects originating from 20 different countries, including 83 males and 17 females, with ages ranging from 16 to 72 years. Unlike several other finger vein datasets that are acquired in fully controlled, darkroom-like environments, the MMCBNU_6000 images were captured in a conventional

office setting, which introduces a realistic degree of variability in illumination and imaging conditions and makes the dataset particularly suitable for evaluating the robustness of recognition systems under near-operational conditions.

For each subject, six fingers were imaged: the index, middle, and ring fingers of both the left and right hand. For each of these six fingers, ten samples were acquired in a single acquisition session, resulting in $100 \times 6 \times 10 = 6,000$ finger vein images in total. Each finger is therefore associated with a distinct identity/class in a closed biometric sense, yielding 600 unique finger classes, each represented by 10 image samples. Images are stored in BMP format, with an original resolution of 640×480 pixels and 8-bit grayscale depth, acquired via near-infrared light transmission imaging of the finger.

The 100 subjects of the database were partitioned according to a two-fold cross-validation protocol at the subject level, ensuring that no subject appears in both the training and test set of the same fold (subject-disjoint evaluation):

- **Fold 1:** subjects 1–50 used for training, subjects 51–100 used for testing;
- **Fold 2:** subjects 51–100 used for training, subjects 1–50 used for testing.

Since each subject contributes six fingers (i.e., six distinct classes), each fold involves 300 identity classes for training and 300 for testing, with all ten samples per finger consistently assigned to the fold-appropriate split. This subject-level partitioning prevents identity leakage between the training and test phases and allows the overall system performance to be reported as the pooled result over both folds, effectively using the entire set of 100 subjects for both training and evaluation across the two complementary runs.

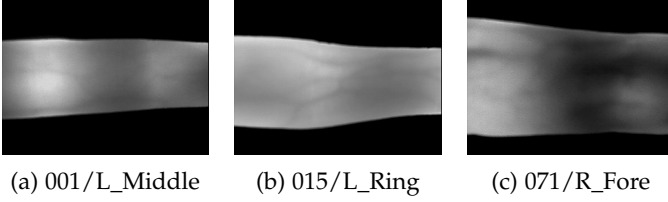


Fig. 1: Acquisition examples from the MMCBNU_6000 dataset

3 METHODOLOGY

This section describes the complete methodology adopted for finger vein recognition, from raw image acquisition to the computation of verification scores. The proposed pipeline is organized into three main stages. First, a dedicated image pre-processing pipeline is applied to each raw finger vein image, encompassing finger boundary detection, geometric alignment, Region of Interest (ROI) extraction and normalization, and contrast enhancement, in order to obtain a clean, well-aligned, and standardized input representation.

Second, the pre-processed images are organized under a subject-disjoint two-fold cross-validation protocol, explicitly designed to evaluate the system under realistic conditions, in which the identities used for testing are never observed during training. Third, two convolutional neural network backbones, ResNet-18 and VGG-16, are trained under an ArcFace angular margin loss to produce discriminative 512-dimensional embeddings, from which genuine and impostor similarity scores are subsequently extracted for verification. This following subsections detail each of these stages.

3.1 Image Pre-Processing

Raw finger vein images acquired by near-infrared imaging devices typically exhibit several sources of variability that can hinder subsequent feature extraction and matching, including finger rotation and translation within the field of view, inconsistent finger width due to differences in positioning, low and non-uniform contrast, and the presence of irrelevant background regions. To address these issues, a dedicated pre-processing pipeline was implemented, consisting of four sequential stages: (i) finger boundary detection, (ii) geometric alignment, (iii) Region of Interest (ROI) extraction and normalization, and (iv) contrast enhancement. All operations were performed on grayscale images.

3.1.1 Image Boundary Detection

The upper and lower boundaries of the finger were estimated using a method inspired by Y. Lu. [3], based on directional edge-response filtering. The input image is first padded along the vertical axis to avoid border artifacts during convolution. Two complementary 4×20 gradient masks are then applied via 2D convolution: one tuned to respond maximally to the upper finger edge (a transition from background to finger, i.e., dark-to-bright along the vertical direction) and one tuned to the lower

edge (bright-to-dark). For each image column, the row position yielding the maximum filter response within the upper and lower half of the image, respectively, is taken as a boundary candidate, and the search is further constrained by excluding responses in the opposite half-plane. Since the raw candidate boundaries are noisy on a pixel level, a median filter with a wide kernel (71 pixels) is applied along the horizontal axis to obtain smooth boundary curves.

To further increase robustness against local artifacts (e.g., due to skin folds, occlusions, or reflections), a consistency check is performed: for the central 50% of the image width, the median boundary position is computed and used as a reference; any column whose estimated boundary deviates from this reference by more than 25% of the reference finger width is replaced by the reference value itself. A second median filtering pass is then applied to yield the final, smooth upper and lower boundary profiles.

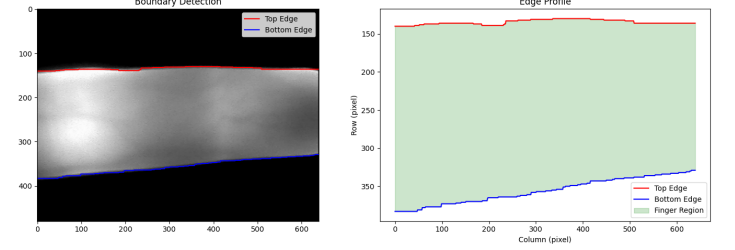


Fig. 2: Boundary Detection

3.1.2 Finger Alignment

Given the estimated boundaries, the finger centerline is computed as the pixel-wise average of the upper and lower boundary curves. A first-order polynomial (linear regression) is fitted to the centerline over the valid image columns, and the slope of the fitted line is converted to an orientation angle expressing the tilt of the finger with respect to the horizontal axis. If the absolute value of this angle exceeds a threshold of 1° , the image is rotated around its center by the corresponding angle using an affine transformation (bicubic interpolation, zero-padding for out-of-bound pixels) in order to horizontally align the finger axis; otherwise, the image is left unaltered. Since rotation modifies the position of the finger boundaries, the boundary detection step described above is re-applied to the aligned image to obtain updated, accurate upper and lower boundary estimates.

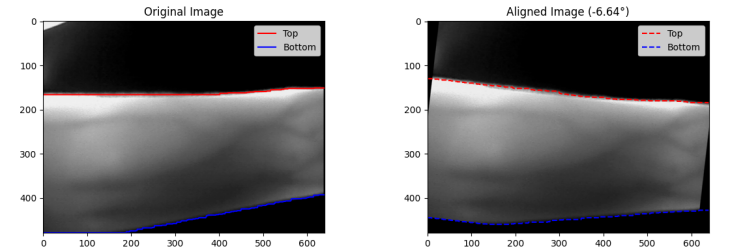


Fig. 3: Finger Alignment

3.1.3 ROI Extraction and Normalization

The Region of Interest is then extracted following a strip-based normalization scheme inspired by Yang et al. [4], which compensates for local variations in finger width and removes background regions surrounding the finger. First, the horizontal extent of the image is cropped to the central 70% (from 15% to 85% of the image width) to discard the finger tip and base, which are more prone to boundary estimation errors. A reference half-width is computed as 40% of the median finger width measured over the central portion of the image (25%–75% of the width), and is used as a fixed sampling half-height for every column of the strip. For each column within the cropped region, a vertical segment centered on the local finger centerline (with the reference half-width) is extracted from the aligned image.

Columns whose pixel intensities are predominantly dark identified via a median-intensity threshold and a minimum-proportion criterion on the number of non-background pixels are discarded, since they typically correspond to background or occluded regions rather than valid finger tissue. Each valid column is then resized, via linear interpolation, to a fixed height, producing a normalized strip in which the finger width is geometrically constant across its entire length. Columns discarded due to the validity criteria are subsequently filled by copying the values of the nearest valid column, ensuring a continuous, artifact-free strip. Finally, the resulting strip is resized via bicubic interpolation to a fixed output resolution of 256×96 pixels, which constitutes the final ROI used for all subsequent processing.

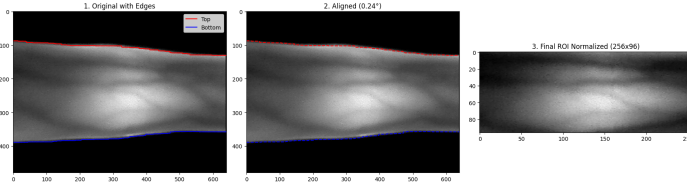


Fig. 4: ROI Extraction

3.1.4 Contrast Enhancement

The extracted ROI is refined through a final enhancement stage aimed at improving vein-pattern visibility while suppressing high-frequency noise. A Gaussian blur (3×3 kernel) is first applied to reduce sensor noise. The resulting image is then rescaled via min-max normalization to fully exploit the 8-bit intensity range [0, 255].

Finally, Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied, with a clip limit of 2.5 and tiles of size 8×8, to locally enhance contrast and emphasize vein structures while limiting the amplification of noise, a known drawback of standard histogram equalization.

The complete pipeline boundary detection, alignment, ROI extraction, and enhancement was applied to every image of the dataset, and the resulting normalized ROIs

were stored to be used as direct input to the subsequent feature extraction and classification stages.

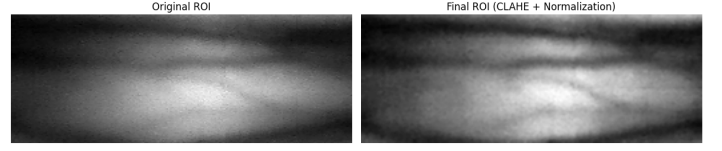


Fig. 5: Applied CLAHE and Normalization to ROI Image

3.2 Data Preparation for Two-Fold Cross-Validation

Before training and evaluating the models, a dedicated data-handling infrastructure was implemented to support a subject-disjoint two-fold cross-validation protocol in which the identities used for testing are never observed during training.

3.2.1 Custom Dataset

A custom *FingerVeinDataset* class, extending PyTorch’s Dataset interface, was implemented to dynamically load only the subset of subjects relevant to a given fold, avoiding the need to physically duplicate data on disk. For each included subject, the six corresponding finger subfolders are iterated in sorted order, and each finger is assigned a unique, incrementally increasing integer class label, with all image samples belonging to that finger inheriting the same label.

These finger-level labels are used exclusively to supervise the training process (via the ArcFace loss); at test time, the trained backbone is used purely as a feature extractor, producing embeddings for previously unseen fingers, which are then compared via similarity scoring rather than classified into a fixed set of known identities. For each fold, two independent *FingerVeinDataset* instances are created one for the training subjects and one for the test subjects each re-deriving its own class indices, since the two subsets involve entirely disjoint sets of finger identities. Given six fingers per subject, each fold involves 300 identity classes for training and 300 held-out identities for testing; the number of training classes is computed dynamically and passed to the ArcFace classification head at training time.

3.2.2 Data Transformation Pipelines

The training pipeline resizes each ROI image to 224×224 pixels to match the input size expected by the pretrained ResNet-18 and VGG-16 backbones, and applies mild data augmentation random rotation of up to ±15° and color jittering of brightness and contrast (±20%) to improve robustness to residual misalignment and illumination variability not fully corrected during pre-processing. Images are then converted to tensors and normalized using standard ImageNet statistics (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]), consistent with the pretrained backbones. The test pipeline applies the same resizing and normalization steps but omits augmentation, ensuring a deterministic and unbiased evaluation.

3.2.3 Pooling Strategy

Since each fold produces an independent set of scores computed on its own held-out subjects, the genuine and impostor similarity scores from both folds are pooled together, separately for each backbone architecture, before computing the final ROC curve, AUC, and EER. This pooling ensures that all 100 subjects of the MMCBNU_6000 database contribute to the evaluation exactly once, across the two complementary folds, maximizing the statistical robustness of the reported results while preserving strict subject disjointness within each individual fold.

3.3 Model Architectures, Training, and Score Extraction

This section describes the deep learning components of the pipeline: the two backbone architectures used as feature extractors, the loss function adopted for training, the training procedure, and the routine used to extract embeddings and compute verification scores.

3.3.1 Backbone Architectures

Two convolutional neural networks, ResNet-18 and VGG-16, were used as feature extraction backbones, both initialized with ImageNet-pretrained weights to leverage transfer learning given the limited size of the training set. Rather than retaining their original classification heads, both architectures were adapted to output a fixed 512-dimensional embedding, consistently with the ArcFace framework described below.

For ResNet-18, the final fully connected layer was replaced with a single linear layer projecting directly to the 512-dimensional space. For VGG-16, the original adaptive pooling was replaced with a global average pooling layer (analogous to ResNet's), and the original three-layer classifier was replaced with a lightweight *Flatten* followed by a single linear layer to the same 512-dimensional output. This redesign reduces the number of trainable parameters in the VGG-16 head and makes the two backbones more directly comparable, while leaving their convolutional feature extractors unchanged.

3.3.2 ArcFace Loss

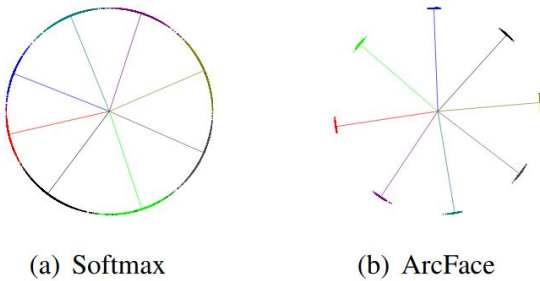


Fig. 6: ArcFace Loss vs Softmax Loss^[1]

Training is supervised using an ArcFace (Additive Angular Margin) loss, implemented as a custom layer. Unlike a standard softmax classifier, ArcFace first L2-normalizes both

the embeddings and the class weight vectors, so that their dot product corresponds to the cosine of the angle between them, and then adds a fixed angular margin ($m = 0.50$) to the angle of the ground-truth class before rescaling all cosine similarities by a factor $s = 64.0$. A numerical safeguard handles the edge case where the angle exceeds the valid margin domain, replacing it with a linear penalty to preserve monotonicity. This formulation pushes the network to learn embeddings that are angularly compact within each identity and well separated across identities.

3.3.3 Training Procedure

For each fold, an ArcFace layer is instantiated with an output size equal to the number of training identities (300), and its parameters are optimized jointly with the backbone using a single Adam optimizer (learning rate 1×10^{-4} , weight decay 1×10^{-4}) and a cosine annealing learning rate scheduler over 20 epochs. At each iteration, a batch of ROI images is forwarded through the backbone to obtain embeddings, which are passed with their labels through the ArcFace layer to compute cross-entropy loss; the loss is backpropagated through both the ArcFace layer and the backbone, and running loss/accuracy are logged after each epoch.

3.3.4 Score Extraction

For each trained model, embeddings and verification scores are computed on the corresponding held-out test subjects. The extracted embeddings are L2-normalized, and pairwise cosine distances are computed across all test samples (upper triangular part only, to avoid duplicate and self-comparisons); genuine/impostor labels are derived by comparing identity labels for each pair, and the similarity score is defined as the negative cosine distance.

4 EVALUATION

Once the two-fold cross-validation cycle described in the previous sections has been completed, the genuine/impostor similarity scores produced independently on each fold are combined through a pooling strategy, and the resulting pooled scores are used to compute the overall verification performance of each backbone architecture. This section describes the evaluation metrics adopted, the procedure used to compute them, and the resulting performance summary.

4.1 Receiver Operating Characteristic (ROC)

For each backbone (ResNet-18 and VGG-16), the pooled arrays of similarity scores and genuine/impostor labels, obtained by concatenating the corresponding results from Fold 1 and Fold 2, are used to compute a Receiver Operating Characteristic (ROC) curve via `sklearn.metrics.roc_curve`, which returns the False Positive Rate (FPR) and True Positive Rate (TPR) across a range of decision thresholds spanning the pooled score distribution.

The resulting pooled ROC curves for both backbones are reported in Figure 7. Both models achieve near-optimal

separability, with ResNet-18 reaching an AUC of 0.9989 and VGG-16 an AUC of 0.9984. Visually, both curves rise almost vertically from the origin and saturate at a Genuine Acceptance Rate close to 1.0 already at very low False Positive Rates (below FAR = 0.05), remaining essentially flat at the maximum value across the rest of the FAR range; this behavior is characteristic of a verification system with very high discriminative power, in which genuine and impostor score distributions exhibit minimal overlap.

Both curves lie almost entirely in the top-left region of the plot, far above the diagonal chance-level line, confirming that the learned embedding spaces of both architectures are highly effective at separating genuine from impostor finger vein comparisons. Comparing the two backbones directly, ResNet-18 slightly outperforms VGG-16 in terms of pooled AUC (0.9989 vs. 0.9984), although the margin between the two curves is minimal and the two ROC curves are visually almost indistinguishable, overlapping closely across the entire operating range. This suggests that, despite their different depth and parameter count, both architectures, once adapted to output a 512-dimensional ArcFace-supervised embedding, are able to learn comparably discriminative representations for this task, with ResNet-18 showing a marginally better trade-off between false acceptances and false rejections at the extreme, near-perfect end of the performance spectrum.

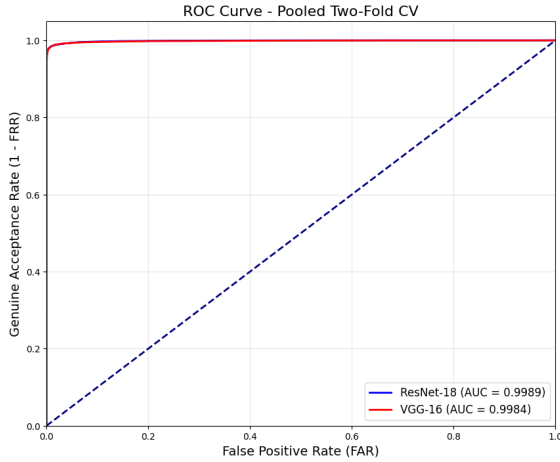


Fig. 7: ROC Curve of Resnet-18 and VGG-16

4.2 Detection Error Trade-off (DET)

Since the pooled ROC curves of the two backbones are visually almost indistinguishable, both saturating near an AUC of ~ 0.999 and offering little room for a fine-grained visual comparison, a Detection Error Trade-off (DET) curve was additionally plotted for both models. The DET curve reports the same underlying error rates as the ROC curve, the False Positive Rate (FAR/FPR) on the x-axis and the False Negative Rate (FRR/FNR) on the y-axis, but on a logarithmic scale for both axes, which has the effect of visually expanding the low-error-rate region.

The resulting DET curves, is showed in Figure 8. A more visible separation emerges in the mid-to-high FAR

range, from approximately 5% to 20%, where the ResNet-18 curve (blue) consistently lies below the VGG-16 curve (orange), indicating that, for any given False Acceptance Rate in this range, ResNet-18 achieves a lower False Rejection Rate than VGG-16 i.e., a consistent advantage in this operating region, where the system is configured to be relatively permissive in accepting matches.

This behavior suggests that, although both models are effectively equivalent at their respective EER operating points, ResNet-18's embedding space provides a slightly better-separated tail of the impostor/genuine score distributions in the higher-FAR regime.

Finally, the two curves terminate at different points along the x-axis: the ResNet-18 curve reaches its minimum plotted FNR (0.1%) already at a FAR of approximately 20–22%, whereas the VGG-16 curve extends further, only reaching the same FNR floor at a FAR of roughly 35–38%. This indicates that, in the extreme high-FAR / low-security regime, ResNet-18's error trade-off curve is more compact and reaches its saturation point sooner, while VGG-16 requires a comparatively higher false acceptance tolerance to drive its false rejection rate down to the same minimal level.

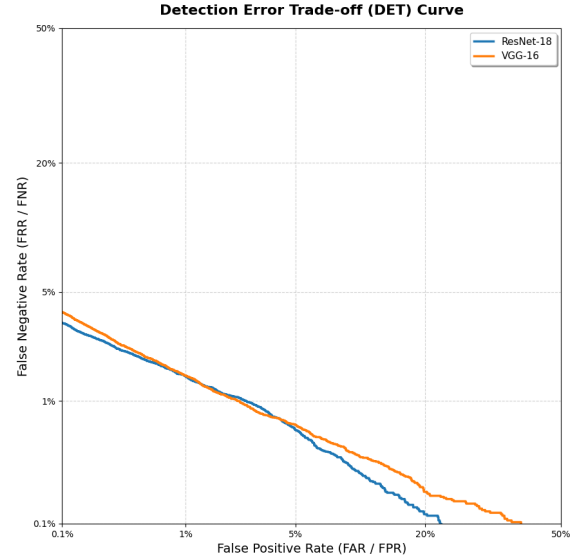


Fig. 8: DET Curve

4.3 Equal Error Rate (EER)

From the same pooled ROC curve, the False Acceptance Rate (FAR) is directly taken as the FPR, while the False Rejection Rate (FRR) is derived as the complement of the TPR ($FRR = 1 - TPR$). The Equal Error Rate is then identified as the operating point at which FAR and FRR are as close as possible, i.e., the threshold minimizing the absolute difference $|FAR - FRR|$. The corresponding EER value and decision threshold are extracted using `np.nanargmin`.

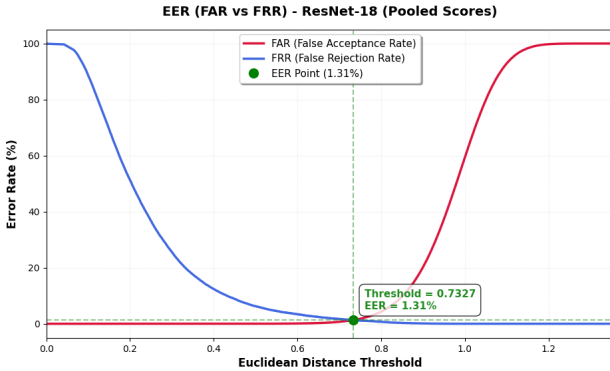
The resulting FAR/FRR curves, plotted as a function of the Euclidean distance threshold, are shown in Figure 9.

Numerically, the two architectures achieve almost identical, very low EER values: 1.31% for ResNet-18, at a distance threshold of 0.7327, and 1.30% for VGG-16, at a distance threshold of 0.7395.

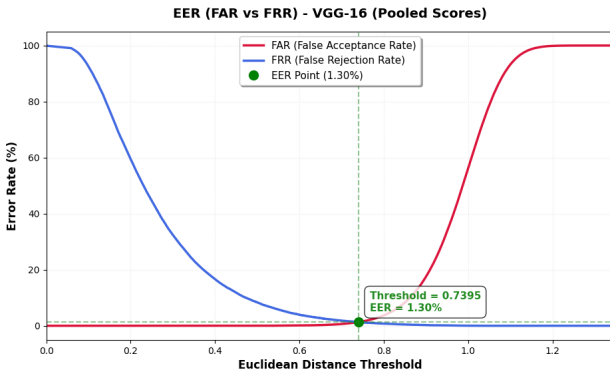
The difference between the two models is minimal (0.01 percentage points) and can be considered practically negligible, confirming what was already observed from the pooled ROC/AUC comparison: VGG-16 attains a marginally lower EER than ResNet-18, despite the latter showing a slightly higher AUC, suggesting that the two models' error trade-off curves cross at a very similar point and that their overall verification performance is, for practical purposes, equivalent on this dataset.

The comparable EER thresholds obtained for both models (0.7327 vs. 0.7395) further indicate that the two embedding spaces, while learned by architecturally different backbones, produce cosine-distance distributions with very similar structure and separation dynamics.

Combined with the extremely low EER values (around 1.3%), these results indicate that the proposed pre-processing pipeline in conjunction with ArcFace-supervised embeddings is highly effective for open-set finger vein verification on the MMCBNU_6000 database, regardless of the specific backbone architecture adopted.



(a) EER ResNet-18



(b) EER VGG-16

Fig. 9: EER of ResNet-18 and VGG-16

TABLE I: VERIFICATION PERFORMANCE

Model	EER (%)	Accuracy (%)	AUC
ResNet-18	1.305	98.69	0.9989
VGG-16	1.304	98.70	0.9984

4.4 Accuracy at the EER Threshold

In addition to the EER itself, a pointwise verification accuracy is computed by applying the identified EER threshold to the pooled similarity scores, classifying each genuine/impostor pair according to whether its score exceeds the threshold, and measuring the proportion of correctly classified pairs against the ground-truth genuine/impostor labels.

At their respective EER operating points, both backbones achieve essentially identical verification accuracy: 98.69% for ResNet-18 and 98.70% for VGG-16, a difference of only 0.01 percentage points. This near-perfect agreement is consistent with the previously observed EER values, since the pointwise accuracy at the EER threshold is, by construction, tightly linked to the balanced error rate.

4.5 Efficiency Metrics

Beyond recognition accuracy, the evaluation also reports two computational efficiency indicators, aggregated consistently with the pooling logic: the average inference time per sample (in milliseconds), obtained by averaging the per-fold inference times measured during score extraction, and the number of trainable parameters (in millions), extracted directly from each backbone at the end of the corresponding fold.

Here, a clear asymmetry emerges between the two backbones, in contrast with their near-identical recognition performance. ResNet-18 is considerably more lightweight, with 11.44 million parameters, compared to 14.98 million for VGG-16, an increase of roughly 31% in model size for the latter, despite the customized, lightweight classification head adopted for VGG-16 in this work.

The gap is even more pronounced in terms of inference speed: ResNet-18 achieves an average inference time of 1.10 ms per sample, whereas VGG-16 requires 4.95 ms per sample, i.e., more than four times slower. This substantial difference in computational cost, combined with the virtually identical EER and accuracy achieved by the two models indicates that ResNet-18 offers a markedly better performance-efficiency trade-off for this task: it matches VGG-16's verification accuracy while requiring fewer parameters and significantly lower inference latency, making it the more practical choice for real-world or resource-constrained deployment scenarios, such as embedded or mobile finger vein recognition systems, without any measurable sacrifice in recognition quality.

TABLE II: COMPUTATIONAL COMPLEXITY AND INFERENCE EFFICIENCY

Model	Parameters (M)	Avg. Inference Time (ms)
ResNet-18	11.44	1.10
VGG-16	14.98	4.95

5 COMPARISON WITH STATE-OF-THE-ART OPEN-SET METHODS

To contextualize the performance achieved by the proposed pipeline, this section compares the obtained results with a selection of deep-learning-based finger-vein recognition methods evaluated under the open-set protocol, as summarized in the review by Hou *et al.* [2].

Table III reports a subset of representative open-set deep-learning-based methods from the literature, together with the results obtained in this work. The reported methods span a variety of architectures and strategies, including conditional GANs for image restoration combined with CNN classification, CNN-based vein pattern recovery, CycleGAN-based vein extraction, composite-image data augmentation, sparse autoencoders, and pretrained VGG-16 models, evaluated on different databases (HKPU, SDU, USM, THU-FVDT2, and proprietary datasets) under different train/test identity splits.

TABLE III: COMPARISON WITH OPEN-SET FINGER-VEIN METHODS

Method	Database	EER (%)
Cond. GAN + CNN	HKPU	1.814
Cond. GAN + CNN	SDU	3.934
CNN recovery	HKPU	2.70
CNN recovery	USM	1.42
CycleGAN	THU-FVDT2	1.12
CycleGAN	SDU	0.94
Composite + CNN	HKPU	0.33
Composite + CNN	SDU	2.35
Sparse AE	SDU	0.70
Pretrained VGG-16	Proprietary	0.396
Pretrained VGG-16	Proprietary	1.275
Pretrained VGG-16	SDU	3.906
GAN aug.	SDU	0.87
ResNet-18 (this work)	MMCBNU_6000	1.305
VGG-16 (this work)	MMCBNU_6000	1.304

As shown in Table III, the EER values reported in the literature for open-set finger-vein verification span a wide range, from as low as 0.33% (composite-image-based CNN on HKPU) and 0.396% (pretrained VGG-16 on a proprietary database) up to 3.9% (pretrained VGG-16 and conditional GAN-based methods on SDU). The results obtained in this work, 1.305% EER for ResNet-18 and 1.304% for VGG-16 on the MMCBNU_6000 database, fall well within this range, positioning the proposed pipeline as competitive with, and in several cases superior to, existing deep-learning-based approaches evaluated under comparable open-set conditions.

It is important to stress that a direct, fully controlled comparison across these methods is not possible, since the reported results were obtained on different databases, with different acquisition conditions, different train/test

identity splits (division ratios), and, in some cases, different numbers of samples per finger; the EER metric, while standardized in its definition, is therefore sensitive to these experimental factors and cannot be interpreted as a perfectly homogeneous benchmark across studies. Nevertheless, the consistency of the results obtained by both ResNet-18 and VGG-16 in this work, evaluated under a subject-disjoint two-fold cross-validation protocol on a large and demographically diverse database such as MMCBNU_6000, provides evidence that the combination of a robust classical pre-processing pipeline with ArcFace-supervised CNN embeddings constitutes an effective and competitive approach for open-set finger-vein verification, in line with the state of the art reported in the literature.

6 CONCLUSIONS

Overall, this project confirms that the combination of a carefully designed pre-processing pipeline with ArcFace-supervised CNN embeddings constitutes a highly effective and architecture-agnostic approach for open-set finger vein verification, achieving sub-2% EER on a demographically diverse, non-darkroom-acquired benchmark such as MMCBNU_6000. Several directions remain open for future work. First, the evaluation could be extended to additional finger vein datasets (e.g., FV-USM, SDUMLA-HMT, HKPU-FV) to assess the generalization capability of the proposed pipeline across different acquisition devices and imaging conditions.

Second, more compact or mobile-oriented architectures (e.g., MobileNet, EfficientNet) could be explored as backbones, potentially achieving an even more favorable performance-efficiency trade-off than ResNet-18. Third, further robustness could be investigated against presentation attacks and adverse acquisition conditions, such as partial occlusions, extreme rotations, or degraded illumination, which are only partially represented in the current dataset. Finally, alternative or hybrid margin-based loss functions could be evaluated to further reduce the residual overlap between genuine and impostor score distributions, particularly in the higher-FAR operating region highlighted by the DET curve analysis.

REFERENCES

- [1] J. Deng, J. Guo, J. Yang, N. Xue, I. Kotsia, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, p. 5962–5979, Oct. 2022. [Online]. Available: <http://dx.doi.org/10.1109/TPAMI.2021.3087709>
- [2] B. Hou, H. Zhang, and R. Yan, "Finger-vein biometric recognition: A review," *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–26, 01 2022.
- [3] Y. Lu, S. J. Xie, S. Yoon, J. Yang, and D. S. Park, "Robust finger vein roi localization based on flexible segmentation," *Sensors*, vol. 13, no. 11, pp. 14 339–14 366, 2013. [Online]. Available: <https://www.mdpi.com/1424-8220/13/11/14339>
- [4] J. Yang, J. Wei, and Y. Shi, "Accurate roi localization and hierarchical hyper-sphere model for finger-vein recognition," *Neurocomputing*, vol. 328, 08 2018.
- [5] S. Y. Z. W. D. S. P. Yu Lu, Shanjuan Xie, "An available database for the research of finger vein recognition."